What is testing

Software testing is basically checking if a software app works the way it should and matches what people asked for.

It helps find problems early so the product ends up being good. I think the main goals are spotting bugs, making sure quality is there, checking if functions do what they are supposed to, and keeping users happy overall.

There's this idea that testing combines verification and validation. Verification is about asking if we are building the thing correctly, like following the plans right. Validation makes sure we are building the actual product that was needed, not something off track.

    - Done by developers
    - Tests individual components
    - Tests interaction between modules
- **System Testing**
    - Tests complete application
- **User Acceptance Testing (UAT)** : Done by client’s/end users

When it comes to types of testing, functional ones look at what the system actually does, making sure it follows the requirements. For example

- **Unit Testing**

unit testing is what developers do on small parts of the code.

- **Integration Testing**

Then integration testing checks how those parts work together. System testing is bigger, looking at the whole app. And user acceptance testing lets the clients or end users try it out to see if it fits.

Non-Functional Testing

Non-functional testing is more about how the system runs, not just what it does. Performance testing sees if its fast enough and responsive.

Load testing checks behavior when lots of users are on it, like normal traffic. Stress testing pushes it to the limits to see what breaks.

Security is for protecting data, and usability is about if users can actually use it without frustration.

It seems like these are important because even if it works, if its slow or hard to use, nobody wants it.

Manual Testing

Manual testing means doing it by hand, without any special tools, just like a real person would interact with it.

That simulates actual use pretty well. Automation testing uses scripts and tools, which is great for stuff that repeats a lot, saving time I guess.

Automation Testing

Automation Testing is the process of using tools and scripts to automatically execute test cases and compare actual results with expected results.

In Automation testing we tools like

UI/UX TOOLS

- Selenium
- Cypress
- Playwright

API TEST TOOL

- Postman
- Rest Assured

MOBILE TEST TOOL

- Appium

There are other kinds too, like

smoke testing for basic checks to see if the main stuff works.

Sanity testing is quick after small changes, just to make sure nothing got messed up.

Regression testing goes back to old features to confirm they still function after updates.

Exploratory testing is more free form, without set test cases, kind of like wandering through to find issues.

SDLC (Software Development Life Cycle)

The software development life cycle, or SDLC, is the whole process to make good software in steps. It starts with gathering requirements, talking to clients about what they want. Then design phase, where they plan the architecture and how it looks, like UI stuff.

Development is when coders build it, writing the features. Testing comes next, with QA finding bugs. After that, deployment to put it live, and maintenance to fix things later or add more.

The flow is requirement to design, then development, testing, deployment, and maintenance looping if needed.

SDLC Flow:

Requirement → Design → Development → Testing → Deployment → Maintenance

STLC (Software Testing Life Cycle)

STLC is specifically for the testing part. It begins with analyzing requirements to see whats testable. Test planning sets the scope, tools, and time.

Designing test cases means writing down how to test. Setting up the environment prepares the setup. Execution runs the tests, and closure evaluates how it went.

Test Case

A test case is like a guide for testing one feature, with details on ID, scenario, preconditions, steps, data, expected and actual results, and pass or fail. For example, take logging in with valid stuff.

Open the page, put in username user123, password pass123, click login, and it should let you in. If it does, status pass. That seems straightforward, but in real cases it gets more detailed.

Phases:

- **Requirement Analysis:** Identify testable requirements
- **Test Planning:** Define scope, tools, timeline
- **Test Case Design:** Write test cases
- **Test Environment Setup:** Prepare testing environment

Bug / Defect

Bugs are when what happens isn't what was expected. The life cycle starts new, gets **assigned, open for work, fixed, retested**, then closed if good. Severity is how bad the bug impacts, like set by the tester.

Priority is how urgent to fix it, decided by managers. They are different, one about damage, other about when to handle.

Key concepts include test scenarios for what to test, test cases for how, and suites as groups of them.

In a real scenario, like testing a login page, positive is valid credentials leading to success.

Negative ones cover wrong password, empty spots, or bad email format. Some people might skip those negatives, but they catch a lot.

Testing really ensures quality in software.

SDLC covers the development side, STLC the testing. Test cases make validation structured.

The sources for learning and research: I drew from are

W3Schools at https://www.w3schools.com,

GeeksforGeeks https://www.geeksforgeeks.org

TutorialsPoint https://www.tutorialspoint.com.

Those sites helped with the basics

For content creation and summarization

ChatGPT at https://chat.openai.com

QuillBot, https://quillbot.com

Notion AI on https://www.notion.so/product/ai

Sometimes it feels like relying on those makes things easier, but I'm not totally sure if I overdid it with the AI part. The research ones seem more straightforward anyway.